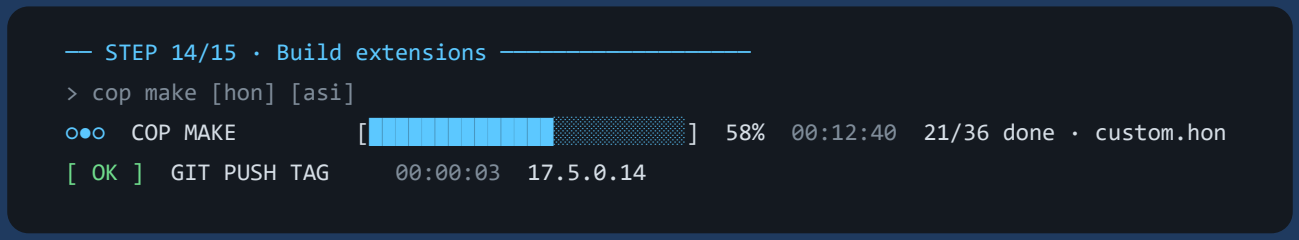
DEVOPS_PYTHON · INTERNAL GUIDE

HNI Build Automation

How the tool works, how it is built, and everything that changed.



PROJECT FOLDER	DevOps_python
WORKSPACE	C:\CetDev\version16.5\extensions\custom
OWNER	Aman Rajesh Choudhari
CODE COVERED	commit 67ad1ca + latest changes (not yet committed)
LAST UPDATED	16 October 2026

Written in simple words for the whole team, with a learning guide for junior developers.

Contents

About this guide 3

Part A · Understanding the tool 4

1. The tool in one page . . . 4
2. Words you will see . . . 5
3. The files and folders . . . 6
4. Architecture: how the parts fit together . . . 7
5. What happens when you run a build . . . 8
6. Dev mode: testing with test branches . . . 10
7. The live progress line (how it works inside) . . . 11
8. Screen design . . . 13

Part B · What changed 15

9. Change history, commit by commit . . . 15
10. Before and after . . . 17

Part C · Open issues and how-to 19

11. Known issues and next steps . . . 19
12. Quick how-to . . . 19

Part D · Learning guide for juniors 21

13. How this code is written (the house style) . . . 21
14. Suggested reading order . . . 23
15. Python ideas used in this project . . . 23
16. Follow one build through the code . . . 25
17. How to make common changes (recipes) . . . 26
18. Practice tasks . . . 27
19. Common mistakes and debugging tips . . . 27

Appendix · Reference 29

A. Function reference . . . 29
B. Every outside command the tool runs . . . 32

About this guide

This guide explains the **HNI Build Automation** tool: what it does, how its parts fit together, how the code is written, and every change made to it since the first version. It uses simple words. You do not need to know Python to follow Parts A, B and C.

Who it is for

- People who **run builds** with the tool.
- People who **look after the code** and change it.
- **Junior developers** who are learning the code. Part D is written for them.

How it is organised

Part	What it covers	Sections
A	What the tool does and how it works inside	1 to 8
B	Every change, commit by commit, and a before/after comparison	9 and 10
C	Open issues and a quick how-to	11 and 12
D	Learning guide for juniors: how the code is written, reading order, Python ideas, recipes, practice tasks	13 to 19
Appendix	Every function and every outside command, in one place	A and B

Code names are printed in this font: `main.py`, `run_beta_build()`, `git pull`. A name ending in () is a function (a named piece of code that does one job).

> **Who wrote what.** The first working version of the tool (commit 418f33e: 13 files, 879 lines) was written completely by Aman. The push steps that came next (commit cfaf195) were also Aman's. Later changes (finishing dev mode, the launcher, the new screen, the live progress line and others) were made by Aman together with Claude, an AI coding assistant. Section 9 says who did what in each change.

PART A

Understanding the tool

What it does, how the parts fit together, and how a build runs.

1. The tool in one page

Making an HNI Beta build by hand takes many commands. You must switch to the right Git branch, download the latest code, clean the workspace, set the new version number, check the changed files, save them with a commit and a tag, send them to the server, build the extensions and upload them. Doing this by hand is slow, and it is easy to forget a step or type a wrong version number.

The HNI Build Automation tool does all of this for you. You answer a few questions. The tool works out the next version number, shows you a summary, and then runs every step in the right order. If any step fails, it stops straight away and tells you why.

You do this	The tool does this
Double-click `run_build.bat`	Opens a console window and starts the tool.
Choose the build type (1 = Beta)	Remembers it. Only Beta is ready today.
Choose the extensions, for example 1,2	Turns them into COP group names like `[hon]` and `[asi]`.
Choose a branch	Checks that the branch really exists on the server copy.
(nothing)	Reads the Git tags and works out the next version, e.g. 16.5.9.11.
Answer y to the summary	Starts the 15 build steps.
Look at the changed files, answer y	Saves them as a commit and creates a tag.
Answer y to push	Sends the commit and tag to the server, builds and uploads.

Note. First Aid and Project Based builds are in the menu, but they are not built yet. Choosing them shows "not implemented yet".

2. Words you will see

These words come up again and again. Each one is explained in one or two simple sentences.

Word	What it means
Repository (repo)	A folder whose full history is kept by Git. The HNI extensions live in the repo at `C:\CetDev\version16.5\extensions\custom`.
Git	The program that keeps track of every change to the code.
Branch	A separate line of work in Git. Each release has its own branch, for example `release/16.5.9`.
origin	The name of the shared copy of the repo on the server (Bitbucket). `origin/release/16.5.9` means "that branch on the server".
Release branch	A branch named `origin/release/<version>`. Used for real builds.
Test branch	A branch named `origin/test/<name>`. Used to try the tool safely (dev mode).
Commit	A saved snapshot of changes, with a short message.
Tag	A permanent name stuck on one commit, for example `16.5.9.11`. The tool uses tags to know which build numbers were already used.
Pull / Push	Pull = download the newest changes from the server. Push = send your commits or tags to the server.
CET	Configura's design software. HNI's extensions run inside it.
COP	The command-line tool of CET Developer (`cop.exe`). It cleans, repairs, sets versions, builds and uploads extensions.
Extension	A plug-in for CET, for example `custom.hon` or `custom.hbf`.
Extension group	A name COP uses for a set of extensions, written in brackets: `[hon]`, `[asi]`, `[gunlocke]`, `[hbf]`, `[hniCore]`.
Build version	The 4-part number given to one build, for example `16.5.9.11` = release `16.5.9` + build number `11`.
Upload name	The name the build gets on the server, for example `16-5-9-11@BETA`.
Dev mode	A switch that makes the tool use test branches instead of release branches.
Console	The black text window (cmd or Windows Terminal) where the tool runs.
Process	A program that is running. The tool starts processes such as `git pull` or `cop make`.
Exit code	A number a program gives back when it ends. 0 means it worked. Anything else means it failed.
Log file	A text file where the full output of every command is saved.

3. The files and folders

This is the whole project. Each folder holds one kind of job.

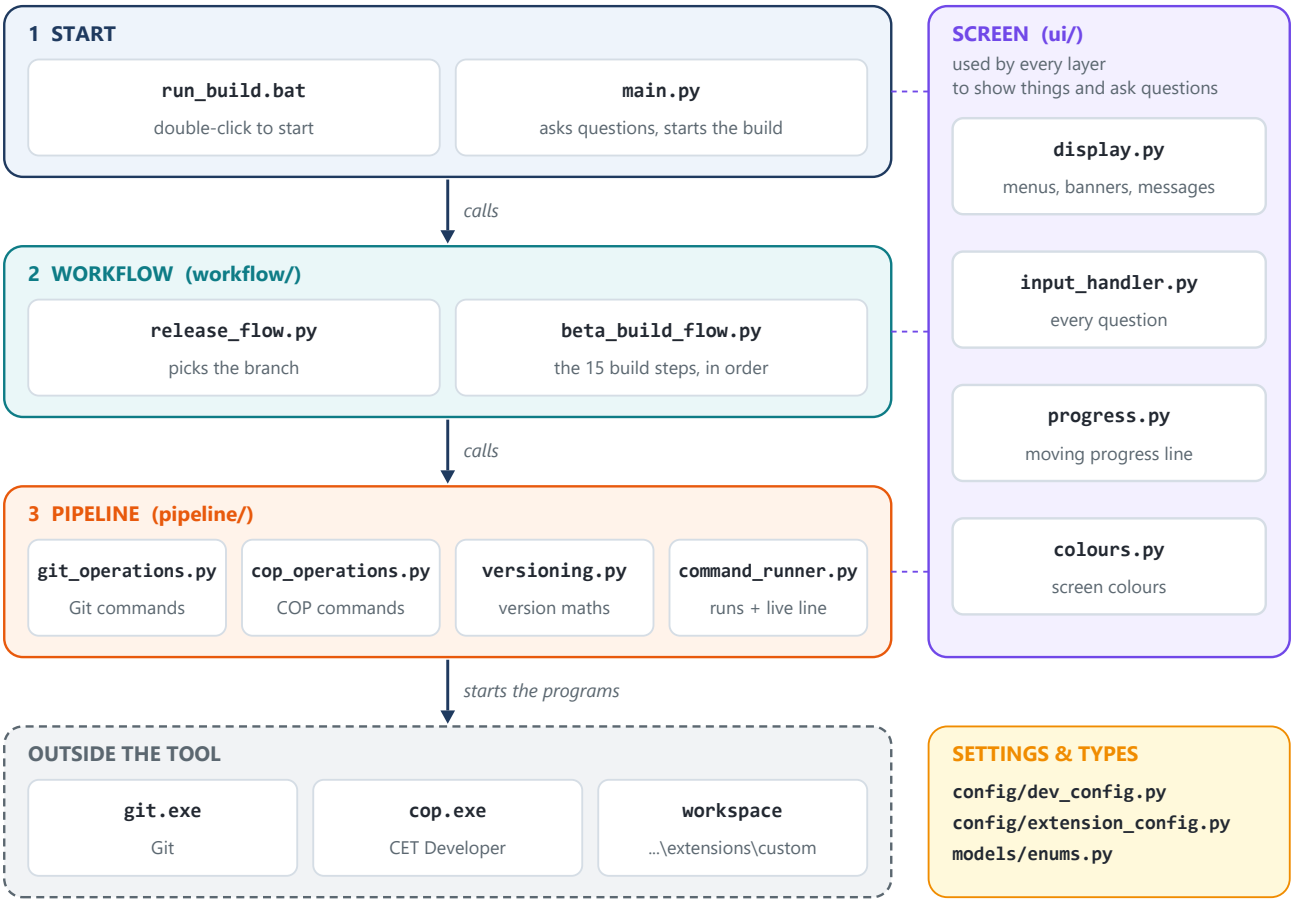
```
DevOps_python/
├── run_build.bat            Double-click this to start the tool
├── main.py                  The starting point of the program
├── config/
│   ├── dev_config.py        The dev mode switch (True / False)
│   └── extension_config.py  Extension -> COP group name, e.g. HON -> [hon]
├── models/
│   └── enums.py             Fixed lists: build types and extensions
├── workflow/
│   ├── release_flow.py      Picks the branch (release or test)
│   └── beta_build_flow.py   The 15 Beta build steps, in order
├── pipeline/
│   ├── git_operations.py    Every Git command
│   ├── cop_operations.py    Every COP command
│   ├── versioning.py        Works out the next version number
│   └── command_runner.py    Runs a command and shows the live progress line
├── ui/
│   ├── display.py           Everything printed on the screen
│   ├── input_handler.py     Every question asked to the user
│   ├── progress.py          The moving progress line and [ OK ] / [FAIL]
│   └── colours.py           Screen colours
├── log/                     Daily log files: build_YYYYMMDD.log
└── utils/                   Empty for now (kept for later)
```

File	Its job, in one sentence	Lines
`run_build.bat`	Opens a console in the right folder, runs main.py and waits at the end so you can read the result.	21
`main.py`	Asks the first questions, works out the version, shows the summary and starts the build.	127
`config/dev_config.py`	Holds one switch: dev_mode. True = test branches, False = release branches.	17
`config/extension_config.py`	Turns each chosen extension into its COP group name.	38
`models/enums.py`	Lists the allowed build types and extensions, so the code never uses loose text for them.	23
`workflow/release_flow.py`	Gets the list of branches, asks which one to use, and checks the answer.	87
`workflow/beta_build_flow.py`	Runs the 15 Beta steps one after another and stops at the first failure.	229
`pipeline/git_operations.py`	Every Git command: list branches and tags, checkout, pull, add, commit, tag, push.	185
`pipeline/cop_operations.py`	Every COP command: clean, repair, setversions, make, upload. One line each.	59
`pipeline/versioning.py`	Pure maths on version numbers: finds the next build number and the upload name.	48
`pipeline/command_runner.py`	Runs one Git or COP command, reads its output live, decides if it worked.	258

File	Its job, in one sentence	Lines
`ui/display.py`	All screens: banner, menus, summary, step headings, success and stopped boxes.	140
`ui/input_handler.py`	All questions. Asks again if the answer is not valid.	144
`ui/progress.py`	Draws the moving progress line and the result lines.	141
`ui/colours.py`	Colour codes and the function that turns colours on in a cmd window.	38

4. Architecture: how the parts fit together

The tool is built in **layers**, like floors in a building. Each layer has one job. A layer only **calls** (uses) the layer below it. The screen code (ui) is used by every layer, but it never makes build decisions itself.



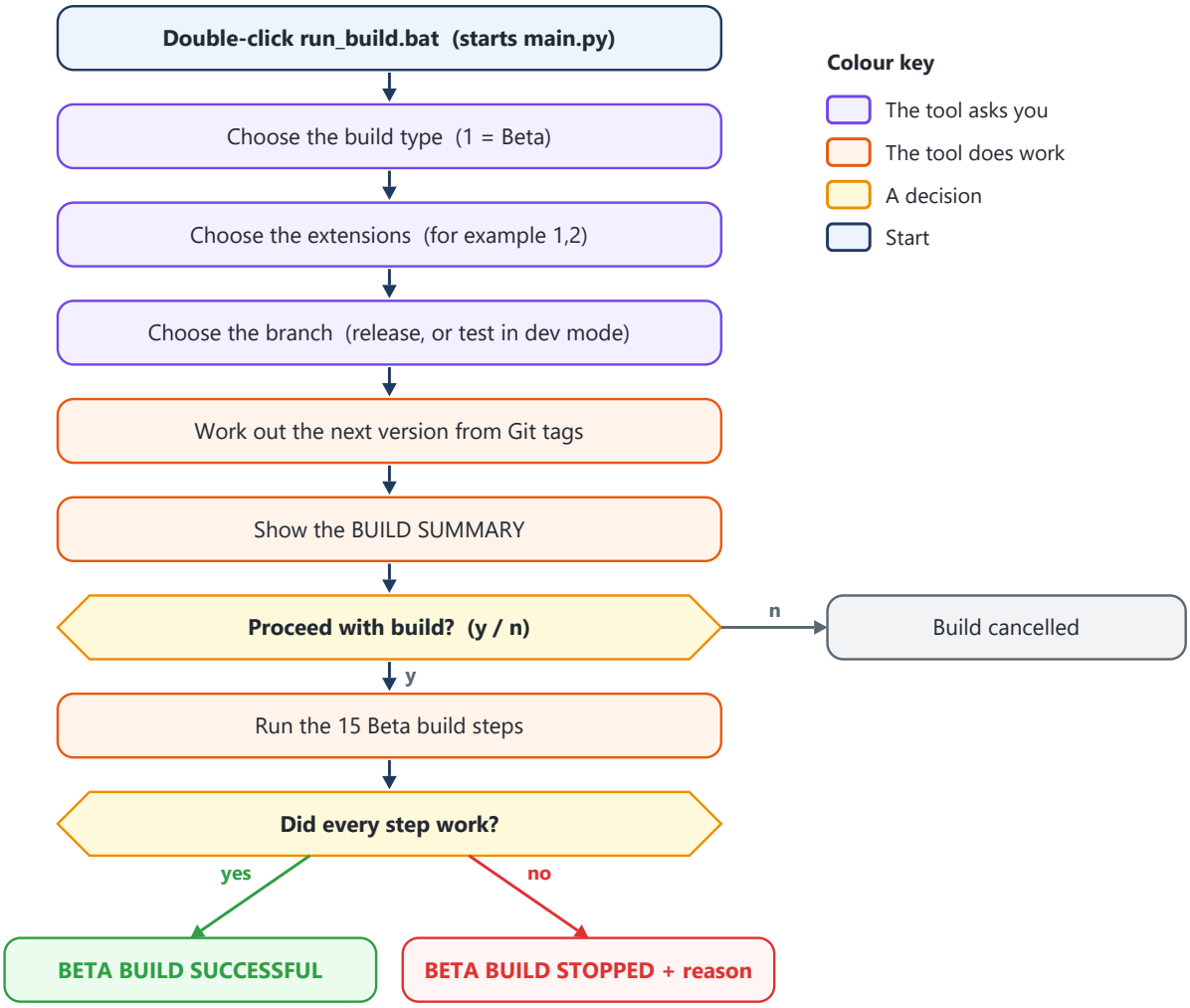
Picture 1. The layers. Solid arrows show which layer calls which. Dashed lines show that every layer uses the screen code.

Layer	Its job	It must NOT
1 Start	Start the tool, ask the first questions, pick which build to run (`main.py`).	Run git or cop commands itself.
2 Workflow	Decide the ORDER of the work and stop when something fails.	Know how a command is typed.
3 Pipeline	Do the real work: run git and cop, do version maths.	Ask the user questions.
Screen (ui)	Show things and ask questions.	Make build decisions.

Layer	Its job	It must NOT
Settings & types	Hold values that rarely change.	Contain logic.

Why it is done this way. When each layer has one job, a change only touches one place. To change how something looks, open ui/. To change the order of the steps, open workflow/. To change a command, open pipeline/. This makes the code easier to read, easier to test and harder to break.

5. What happens when you run a build



Picture 2. The journey from double-click to result.

5.1 Starting

`run_build.bat` moves to the project folder, runs `python main.py` and pauses at the end so the window stays open. `main()` first turns on colours, then prints the banner. In dev mode it also prints a yellow `[DEV MODE]` line.

5.2 The questions

- **Build type**: 1 Beta, 2 First Aid, 3 Project Based. You can type the number or the name.
- **Extensions**: 1 HON, 2 Allsteel, 3 Gunlocke, 4 HBF, 5 HNI-Core. Separate several with commas: 1,2,5.
- **Branch** (normal mode): **l** = Latest (the release branch with the highest number) or **c** = Custom (you type one). You may type `origin/release/16.5.9`, `release/16.5.9` or just `16.5.9`.

- **Branch** (dev mode): the tool lists the test branches and you type one (see section 6).

If an answer is not valid, the tool prints a yellow message and asks the same question again.

5.3 Working out the version number

This happens in `pipeline/versioning.py`. It uses the Git tags to find the next free build number.

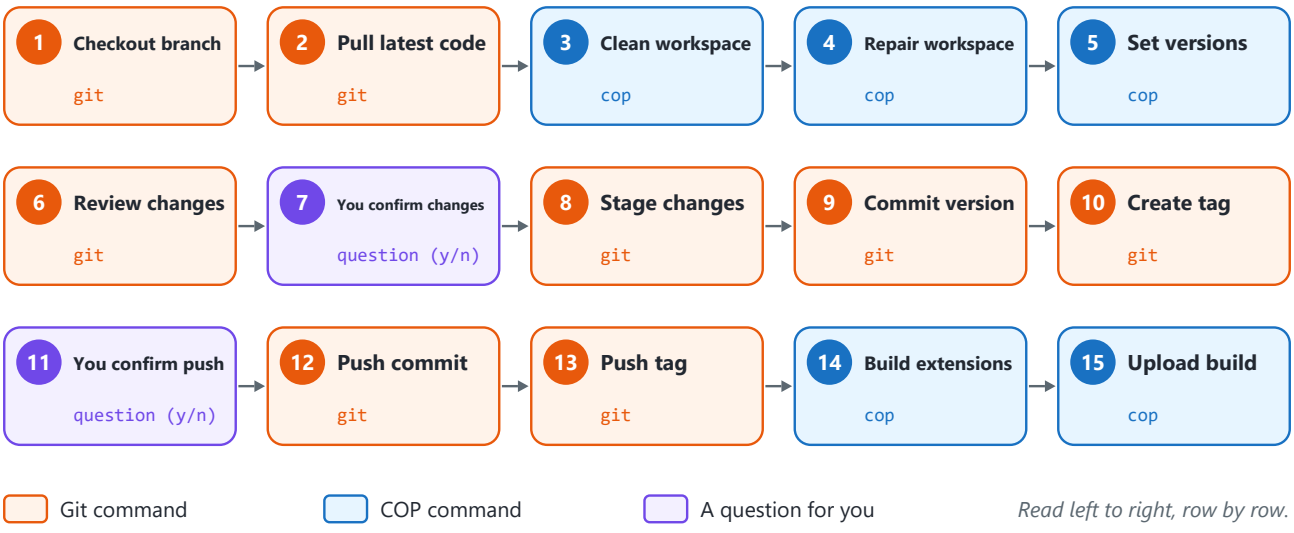
Step	What the tool does	Example
1	Takes the last part of the branch name.	`origin/release/16.5.9` → `16.5.9`
2	Reads every tag in the repo (`git tag --list`).	hundreds of tags
3	Keeps only the tags that start with the version and a dot.	`16.5.9.8`, `16.5.9.9`, `16.5.9.10`
4	Finds the biggest last number.	`10`
5	Adds one. That is the new build version.	`16.5.9.11`
6	Makes the upload name: dots become dashes, @BETA is added.	`16-5-9-11@BETA`

> **Note.** If there are no tags for that version yet, the build number starts at 1. The tool reads tags from your local copy only. See issue 2 in section 11.

5.4 Summary and confirmation

The tool shows the build type, extensions, branch, release version and the new version (in green). Nothing has been changed yet. If you answer n, the tool stops and nothing happens.

5.5 The 15 Beta build steps



Picture 3. The 15 steps. Orange runs Git, blue runs COP, purple waits for your answer.

#	Step	Command that runs	Notes
1	Checkout branch	`git checkout <branch>`	`origin/` is removed from the name first.
2	Pull latest code	`git pull --progress`	--progress gives a real progress bar.
3	Clean workspace	`cop clean`	Stops running CET processes and cleans the write folder.

#	Step	Command that runs	Notes
4	Repair workspace	`cop repair`	Removes settings and temporary files.
5	Set versions	`cop setversions [groups] /version=X /notag /nocommit`	Writes the new version into the extension files. COP does not commit or tag; the tool does that later.
6	Review changes	`git status --short`	Shows the changed files in colour. Stops if nothing changed.
7	You confirm	(question)	Are these changes correct and safe to commit?
8	Stage changes	`git add .`	Marks all changes to be committed.
9	Commit version	`git commit -m <version>`	The commit message is the version, e.g. 16.5.9.11.
10	Create tag	`git tag <version>`	Marks this commit with the version.
11	You confirm push	(question)	Are these changes correct and safe to push?
12	Push commit	`git push --progress`	Sends the commit to the server.
13	Push tag	`git push --progress origin <version>`	Sends the tag to the server.
14	Build extensions	`cop make [groups]`	The long step. The bar fills as each extension finishes.
15	Upload build	`cop upload [groups] /uploadname=<name>`	Uploads to Build Center.

> **Watch out.** Steps 12 and 13 send your commit and tag to the **shared server**. After that, everyone can see them, and they are hard to take back. That is why the tool asks you first (step 11).

5.6 The end

If every step works, you see a green **BETA BUILD SUCCESSFUL** box with the build version, upload name and total time. If a step fails, the tool stops at once, prints a red **BETA BUILD STOPPED** box with the reason, and does not run the steps after it.

6. Dev mode: testing with test branches

Dev mode lets you try the whole build on a **test branch** instead of a real release branch. It is switched on and off in one place:

```
# config/dev_config.py
dev_mode = True      # True  = use origin/test/*    branches (testing)
                     # False = use origin/release/* branches (real builds)
```

What	Normal mode	Dev mode
Banner	normal	extra yellow [DEV MODE] line
Branch list	`origin/release/*`	`origin/test/*`
How you choose	Latest or Custom	The list is shown; you type one

What	Normal mode	Dev mode
What you may type	`origin/release/16.5.9`, `release/16.5.9`, `16.5.9`	`origin/test/demo.build.17.5.0`, `test/demo.build.17.5.0`, `demo.build.17.5.0`
Version comes from	the last part of the branch name	only the numbers in the last part: `demo.build.17.5.0` → `17.5.0`
The 15 steps	all run	all run for real, including push, tag push and upload

Watch out. Dev mode does **not** make the build pretend. It really pushes to the test branch, really pushes the tag and really uploads. Tags are shared by the whole repo, so a test branch should use a version number that no real release uses, for example test/demo.build.99.0.0. See issue 1 in section 11.

All code that exists only for dev mode is marked with a `# Test` comment, so it is easy to find later.

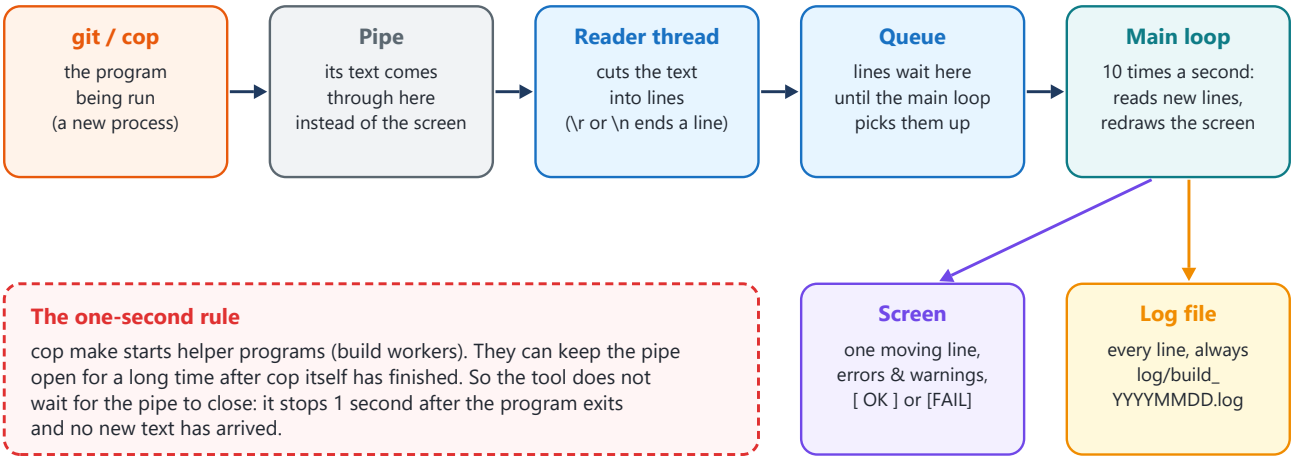
7. The live progress line (how it works inside)

7.1 The problem it solves

Git and COP print a lot of text. COP also draws its own status by moving the cursor around the console. When another program (our tool) reads COP's output, COP cannot do that, so the screen filled up with many repeated "Work in progress" lines. Long steps like cop make also looked frozen for many minutes.

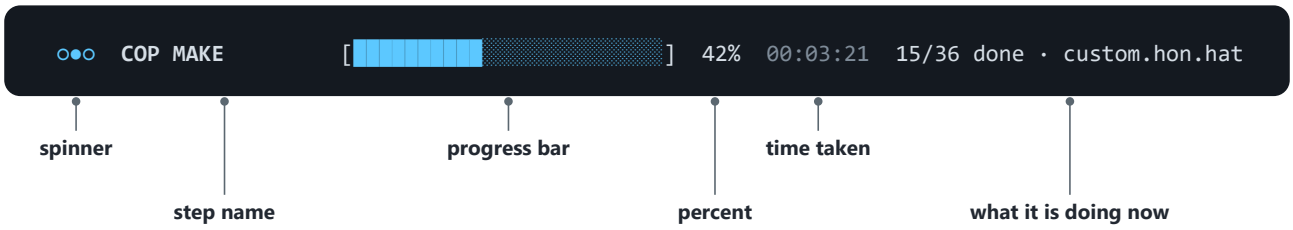
7.2 The idea

One function, `run_command_live()` in `pipeline/command_runner.py`, runs every Git and COP step. It reads the program's text itself and shows **one moving line** instead. When the program ends, the moving line is replaced by one result line: `[ OK ]` or `[FAIL]`. Important text (errors and warnings) is still printed in full. Everything else is saved to the log file.



Picture 4. Inside run_command_live().

7.3 Reading the moving line



- **Spinner**: always moves, so you can see the tool is alive.
- **Progress bar** and **percent**: how far the step is (when this is known).
- **Time taken**: time since this step started.
- **What it is doing now**: the latest thing the program said, cut short so the line never wraps.

7.4 Where the percent comes from

Command	How the percent is found
`cop make`	COP prints the list of targets first, then one line each time a target finishes (for example `* Warning custom.hbf`). Percent = finished targets ÷ all targets.
`git pull / git push`	Git prints its own percent, for example `Receiving objects: 45%`.
COP progress lines	If COP prints `Work in progress 42% | ...`, that percent is used.
Anything else	The percent is not known. A block bounces left and right in the bar instead: it means "working".

7.5 How the tool decides if a step worked

A step **fails** if any of these is true:

- The program's exit code is not 0.
- The COP make summary says some items failed, e.g. `1 failed`.
- A COP make target finished with `Errors`.
- COP printed a known failure message even with exit code 0: `No files to upload`, `Upload finished with errors`, `Upload completed with errors`, `Error: not all files were successfully uploaded`.

7.6 What you see when a step fails

- The red [FAIL] line with the reason.
- The failed COP targets, each with the path to its own build log.
- The last 15 lines the program printed.
- The COP log path (if COP gave one) and the path of the full log file.
- If COP needed a login it could not ask for: a yellow hint to run `cop login` once in a normal terminal.

7.7 The log file

The full output of every command is added to `log/build_YYYYMMDD.log` (one file per day). Each command starts with a line like `===== 18:49:08 cop clean =====`. Git ignores these files because `.gitignore` contains `*.log`.

7.8 Small technical details (for people who maintain it)

- A helper **thread** (`read_output()`) reads the text and puts each line in a **queue**. The main loop wakes up 10 times a second, takes the new lines and redraws the screen. This way the spinner keeps moving even when the program is quiet.
- A line can end with \n (new line) or \r (go back to the start of the line). Both count as the end of a line.
- Text is read as UTF-8; bad characters are replaced instead of crashing.
- The one-second rule (picture 4) stops the tool from waiting forever for COP's build workers.
- If the output is not going to a real console (for example it is sent to a file), the animation switches itself off (`can_animate()`).

8. Screen design

These are the screens the tool shows. The colours are the real ones.

```
╔═══════════════════════════════════════╗
║     H N I   B U I L D   A U T O M A T I O N     ║
╚═══════════════════════════════════════╝
[DEV MODE] Building from origin/test/* branches

── SELECT BUILD TYPE ──────────────────────────
  [1] Beta
  [2] First Aid
  [3] Project Based

? Enter choice: :) 1

── TEST BRANCHES ──────────────────────────────
  • origin/test/demo.build.17.5.0

? Enter test branch: demo.build.17.5.0

── BUILD SUMMARY ──────────────────────────────
  Build Type       BETA
  Extensions       HBF
  Release Branch   origin/test/demo.build.17.5.0
  Release Version  17.5.0
  Next Version     17.5.0.14
────────────────────────────────────────────────

? Proceed with build? [y/n]: y
```

Screen 1. Start, questions and the build summary (dev mode).

```
── STEP 2/15 · Pull latest source ──────────────────
> git pull --progress
○●○  GIT PULL        [████████▒▒▒▒▒▒▒▒▒▒▒▒]  40%  00:00:01  Receiving objects

── STEP 6/15 · Review Git changes ──────────────────
   M  custom/hbf/extension.xml
   M  custom/hbf/version.cm
  2 file(s) changed

? Are these changes correct and safe to commit? [y/n]: y

── STEP 14/15 · Build extensions ───────────────────
> cop make [hbf]
  Warning  custom.hbf                        (worker1, time: 00:01:04)
[ OK ]  COP MAKE         00:01:23  1 item(s), 0 succeeded, 1 with warnings, 0 failed, 0 skipped

╔═══════════════════════════════════════════════════╗
║              BETA BUILD SUCCESSFUL                ║
╚═══════════════════════════════════════════════════╝

  Build Version    17.5.0.14
  Upload Name      17-5-0-14@BETA
  Total Time       00:24:10
```

Screen 2. Steps running (file names are examples). The GIT PULL line is shown while it moves; when a step ends it becomes one [OK] line.

```
── STEP 15/15 · Upload Beta build ──────────────────
> cop upload [hbf] /uploadname=17-5-0-14@BETA
[FAIL]  COP UPLOAD       00:00:09  No files to upload

  Last output:
  Logging in to MyConfigura...
  Successfully logged in to MyConfigura.
  No files to upload! Please make sure the built extension versions match ...

  Full output     : C:\...\DevOps_python\log\build_20261016.log

╔═══════════════════════════════════════════════════╗
║               BETA BUILD STOPPED                  ║
╚═══════════════════════════════════════════════════╝

  Reason           Failed during COP upload.
```

Screen 3. A failed step. This is a real message COP gave on 16 October.

Colour / sign	Meaning
Cyan	Headings, menus, the progress bar and the spinner.
Green	Success: [OK], the new version number, the success box.
Red	Failure: [FAIL], errors, the stopped box.
Yellow	Warnings, wrong answers, dev mode.
Grey (dim)	Less important details: the command being run, times, file paths.
?	A question for you.
!	A wrong answer or a warning.
x	An error that ends the tool.

Why it is done this way. Lines, boxes, the bar and the spinner (= ‖ – ▒ • ○) are characters that exist in the normal console font, so they also look right in a plain cmd window. `enable_console_colours()` in `ui/colours.py` turns colours on at the start; older cmd windows need this.

PART B

What changed

Every commit from the first line to today, and a before/after comparison.

9. Change history, commit by commit

The project has 6 commits plus a set of changes that were not committed yet when this guide was written. Oldest first.

Commit	Date	Title	By
`10d1bb7`	14 Oct 2026	Initial commit	Aman
`418f33e`	14 Oct 2026	implemented HNI Devops Repo	Aman
`cfaf195`	15 Oct 2026	Added git push_build_tag and push_changes	Aman
`a529484`	15 Oct 2026	added dev mode	Aman, finished with Claude
`b5c64bc`	16 Oct 2026	Added run utility, fixed cop_make	Aman with Claude
`67ad1ca`	16 Oct 2026	Login dependency removed (see 9.6)	Aman with Claude
(working copy)	16 Oct 2026	New progress line, git steps, new screen, login removed	Aman with Claude

9.1 10d1bb7 · Initial commit

14 Oct 2026 · Aman · 1 file

Created the repository with a standard Python `.gitignore`.

- It already ignores `*.log` files. The daily log files added much later rely on this.

9.2 418f33e · implemented HNI Devops Repo (the first version)

14 Oct 2026 · written completely by Aman · 13 files · 879 lines

This commit is the whole first working tool. Everything later is built on top of it.

- The folder structure and the layer idea: `config`, `models`, `pipeline`, `ui`, `workflow` (plus empty `log` and `utils`).
- Menus for build type and extensions. Wrong answers are asked again.
- Enums for build types and extensions (`models/enums.py`), and the extension → COP group map.
- Release branch choice: Latest (highest version) or Custom (checked against the server list).
- Version maths from Git tags (`pipeline/versioning.py`) and the upload name.
- The build summary before starting.
- The Beta flow: checkout, pull, cop clean, cop repair, cop setversions, show changed files, confirm, git add, commit, tag, cop make, cop upload (12 steps).

- The house style used in every file since: header blocks, one job per function, True/False results, numbered step comments.

> **Why it is done this way.** Every later change kept this structure. Because the first version already split the work into layers, new features could be added without rewriting it.

9.3 cfaf195 · Added git push_build_tag and push_changes

15 Oct 2026 · Aman · 4 files

- New `push_changes()` (`git push`) and `push_build_tag()` (`git push origin <tag>`).
- A second question before pushing. The Beta flow grew from 12 to 15 steps.
- COP commands now print the command and show their output in the console (before, the output was hidden until the end).
- Each COP step got a heading line. It had a small typo, `/n` instead of `\n`, fixed in 9.5.

9.4 a529484 · added dev mode

15 Oct 2026 · started by Aman, finished with Claude · 7 files

Aman started dev mode: the flag, the list of test branches, the test-branch question and a first live version of cop make that redrew "Work in progress" lines. It was finished together with Claude:

- `config/dev_config.py`: one place for the `dev_mode` switch. It had been written in two files, which could disagree.
- `validate_test_branch()`: accepts `origin/test/x`, `test/x` or `x`, and keeps the real upper/lower case.
- `get_test_version_string_from_branch()`: reads `17.5.0` out of `test/demo.build.17.5.0`. Without it the version became `demo.build.17.5.0` and cop setversions would fail.
- Bug fixed: a wrong test-branch answer jumped to the release-branch question.
- The test branches are listed before the question, and a dev mode line is shown at the start.
- All test-only code is marked with `# Test`.

9.5 b5c64bc · Added run utility, fixed cop_make

16 Oct 2026 · Aman with Claude · 3 files

- New `run_build.bat`: double-click to start the tool in a normal console window.
- cop make was changed to write straight to the console, the same way Aman's working .bat file ran it, with a heading, a result block with the time taken, and a short error list (`/printLastErrorsCompact`) on failure.
- The `/n` typo in the COP headings was fixed.

> **Why it is done this way.** At this point the tool still printed COP's own output. It was clean in some consoles but messy in others. That led to the live progress line in 9.7.

9.6 67ad1ca · "Login dependency removed"

16 Oct 2026 · Aman with Claude · 4 files

Careful: the name is misleading. This commit **added** a login step. It is removed again in 9.7.

- COP asked for the MyConfigura login many times during one build.
- The new step 0 asked once for e-mail and password (the password hidden while typing) and passed them to COP through the `CETDEV_MC_USERNAME` / `CETDEV_MC_PASSWORD` environment variables. Nothing was saved to disk.

> **Why it is done this way.** It was removed in 9.7 because COP turned out to log in by itself with its saved login (the real COP log shows "Logging in to MyConfigura using e-mail: ...").

9.7 Latest changes (working copy, not committed yet)

16 Oct 2026 · Aman with Claude · 7 files changed, 3 new files

The biggest change after the first version. It made the screen clean and alive, and made failures easier to understand.

- **Login step removed** again (step 0, the question, the helper functions). If COP ever needs a login, the tool now says "run `cop login` once".
- **New** `pipeline/command_runner.py`: `run_command_live()` runs every Git and COP step with one moving progress line.
- **New** `ui/progress.py`: draws the moving line, the [OK] / [FAIL] lines and the failure details.
- **New** `ui/colours.py`: colour codes and `enable_console_colours()`.
- **Git steps** (checkout, pull, add, commit, tag, push, push tag) now use the same runner. Pull and push use --progress for a real bar.
- `cop_operations.py` became very short: every COP command is one call to `run_command_live()`.
- **New screen**: boxed banner, section headings, coloured menus and prompts, coloured changed-file list, "STEP 3/15" headings, success and stopped boxes with the total time.
- **Better failure checks**: a COP upload that says "No files to upload" is now a failure, even though COP's exit code is 0.
- **Logs**: full output of every command in `log/build_YYYYMMDD.log`.
- Step comments 11 to 13 in beta_build_flow.py were corrected (they all said "Commit build version").

> **Why it is done this way.** The real COP logs from 16 October were replayed through the new code to check it. They showed that COP prints no percent when its output is read by another program, so for cop make the bar is filled from finished targets instead.

10. Before and after

Area	First version (418f33e)	Now
Starting	`python main.py`	Double-click `run_build.bat` (or python main.py)
Branches	release branches only	release branches, or test branches in dev mode
Pushing	not done	commit and tag pushed after a yes/no question
COP output	hidden until the end	one moving line; errors and warnings shown in full
Git output	hidden; one short message	the same moving line; real percent for pull and push
Long steps	no sign of life	spinner, bar, timer and the current target
Failures	a short message	red box with the reason, failed targets and their logs, last 15 lines
Upload with no files	counted as success	counted as failure
Logs	none	`log/build_YYYYMMDD.log` with everything

Area	First version (418f33e)	Now
Look	plain text	colours, boxes, numbered steps
Login	COP asked, sometimes many times	COP uses its saved login; the tool explains if a login is needed

PART C

Open issues and how-to

What still needs attention, and quick answers for everyday use.

11. Known issues and next steps

#	Issue	What it means	What to do
1	Test tags use real release numbers	Test builds on `test/demo.build.17.5.0` pushed tags 17.5.0.2, 17.5.0.3 and 17.5.0.4 to the server. But `origin/release/17.5.0` is a real release branch, so real 17.5.0 builds will count these tags.	Give test branches a version no release uses (e.g. 99.0.0), or add a prefix to dev mode tags. Agree with the team before deleting any tags from the server.
2	Tags are not downloaded before the version is worked out	The tool reads tags from your local copy only. On 16 October the server had 17.5.0.5 but the local copy did not. The tag push then fails after the commit is already pushed.	Run `git fetch --tags` before a build. Better: add a fetch step to the tool (practice task 3).
3	Upload found no files	In the run on 16 October at 18:52, cop upload said "No files to upload!". The cause is not known yet. One difference from the working .bat file: it runs COP from `...\extensions`, the tool from `...\extensions\custom`.	Look into it. The tool now at least reports it as a failure.
4	Dev mode is switched on	`config/dev_config.py` says `dev_mode = True`.	Set it to False before real release builds.
5	The workspace path is fixed in the code	`main.py`: `repo_path = C:\CetDev\version16.5\extensions\custom`.	Move it to config/ if other workspaces are needed.
6	Two build types are not built	First Aid and Project Based show "not implemented yet".	Add a flow for each (recipe D in section 17).
7	The newest screen has not run a full real build yet	It was tested with fake commands and by replaying real COP logs from 16 October.	Watch the first real run. The full output is in log/.

12. Quick how-to

I want to...	Do this
start a build	Double-click `run_build.bat`. Answer the questions.
do a real build, not a test	Open `config/dev_config.py` and set `dev_mode = False`.
avoid a version clash	In `C:\CetDev\version16.5\extensions\custom` run `git fetch --tags` before the build.
understand a failed step	Read the red box. Then read "Last output". Then open the log file it names. For cop make, open the failed target's build log.

I want to…	Do this
see COP's own log	Run `cop show latest log` in the workspace.
find old output	Open `log/build_YYYYMMDD.log` for that day.
fix strange characters or no colours	Run the tool from a cmd window or Windows Terminal (run_build.bat does this).
run one command by hand	Copy the grey `> ...` line under the step heading and run it in a cmd window in the workspace.

PART D

Learning guide for juniors

How the code is written, how to read it, and how to change it safely.

This part is for new developers. It explains how the code is written and why, in the order you would learn it. Read it with the code open next to you.

13. How this code is written (the house style)

Aman wrote the first version with a few simple rules. Every later change follows the same rules. If you follow them too, your code will fit in.

Rule 1. Every file starts with a header block

```
# ============================================================
# Project      : HNI Build Automation
# File         : versioning.py
# Author       : Aman Rajesh Choudhari
# Created      : 14-Oct-2026
#
# Description:
# Handles build-version calculation for the HNI build
# automation pipeline.
# ============================================================
```

The header tells the reader what the file is for before they read any code.

Rule 2. One folder = one kind of job. One function = one small job

Git commands live in `git_operations.py`, questions in `input_handler.py`, printing in `display.py`. Each function does one thing and its name says what:

```
def get_next_build_version(release_version: str, latest_build_number: int) -> str:
    next_build_number = latest_build_number + 1
    return f"{release_version}.{next_build_number}"
```

Rule 3. Work functions return True or False, and the caller stops early

A function that runs a step returns True if it worked and False if it failed. The workflow checks the answer and stops at the first failure:

```
if not cop_clean(repo_path):
    show_build_stopped("Stopped during COP clean.")
    return False
```

Why it is done this way. If a step failed and the build went on, the tool could push or upload a broken build. Stopping early keeps the server safe.

Rule 4. Questions repeat until the answer is valid

When the answer is wrong, the function simply calls itself again. This is called **recursion**.

```
def get_build_confirmation() -> bool:
    user_input = input(prompt_text("Proceed with build? [y/n]: "))
    user_input = user_input.lower().strip()
    if user_input == "y" or user_input == "yes":
        return True
    elif user_input == "n" or user_input == "no":
        return False
    show_input_error_msg("Incorrect input.")
    return get_build_confirmation()      # ask again
```

Rule 5. Fixed choices are enums, not loose text

```
class Build_Type(Enum):
    NONE=0
    BETA=1
    FIRST_AID=2
    PROJECT_BASED=3
```

Writing `Build_Type.BETA` is safer than writing "beta": a typo becomes an error you see at once, not a silent bug. `NONE` means "no valid choice yet".

Rule 6. Showing is separate from doing

Printing lives in `ui/`. Work lives in `pipeline/`. A pipeline function never calls `print()` for normal messages; it uses the runner or a ui function. This keeps the screen clean.

Rule 7. Big steps get numbered comment banners

```
    # ------------------------------------------------------
    # 3. Clean workspace
    # ------------------------------------------------------
```

Rule 8. Test-only code is marked

Every line that exists only for dev mode has a `# Test` comment above it. Search for it to find all of them.

Rule 9. Names follow a pattern

Name starts with	It means	Example
`get_`	returns a value, usually after asking or reading something	`get_git_tags()`
`show_`	prints something	`show_build_summary()`
`ask_..._msg`	prints the text of a question or menu	`ask_build_type_msg()`
`validate_`	checks a value; returns it cleaned up, or None	`validate_release_branch()`
`cop_`	runs one COP command	`cop_make()`
`run_`	runs a whole process or command	`run_beta_build()`

Rule 10. Type hints say what goes in and what comes out

```
def validate_release_branch(branch: str, release_branch_list: list[str]) -> str | None:
```

Read it as: "takes a text and a list of texts, gives back a text or None". `str | None` means "either a text, or nothing if the branch is not valid".

14. Suggested reading order

Read the files in this order. Each one uses ideas from the ones before it. The times are a guide for a first read.

#	File	What to look for	Time
1	`models/enums.py`	What an Enum is.	5 min
2	`config/extension_config.py`	Turning an enum into a COP group name with if / elif.	5 min
3	`pipeline/versioning.py`	Pure functions: values in, answer out. Easy to test.	10 min
4	`pipeline/git_operations.py`	The top half: reading branches and tags with `subprocess.run`.	15 min
5	`ui/display.py` + `ui/input_handler.py`	Menus and questions; asking again with recursion.	15 min
6	`workflow/release_flow.py`	Putting the pieces together to choose a branch.	10 min
7	`main.py`	The whole story from start to the summary.	10 min
8	`workflow/beta_build_flow.py`	The 15 steps and the stop-early pattern.	15 min
9	`pipeline/cop_operations.py`	Each COP command is one line.	5 min
10	`pipeline/command_runner.py` + `ui/progress.py`	The advanced part: processes, threads, queues, the live line.	30 min

15. Python ideas used in this project

Each idea below is shown with real code from this project.

Functions and return values

A function is a named piece of code. It can take inputs (parameters) and give back a result with return.

```
def get_beta_upload_name(build_version: str) -> str:
    return build_version.replace(".", "-") + "@BETA"
# get_beta_upload_name("16.5.9.11")  ->  "16-5-9-11@BETA"
```

Lists and loops

A list holds many values in order. A for loop goes through them one by one.

```
release_tags = []
for tag in tags:
    tag = tag.strip()
    if tag.startswith(prefix):
        release_tags.append(tag)
```

Dictionaries

A dictionary links a key to a value, like a phone book. The runner uses one to remember each target's log file.

```
target_logs = {}
target_logs["custom.hbf"] = "C:\\...\\custom.hbf....buildLog.cmtxt"
```

Text (string) tools

strip() removes spaces at the ends, lower() makes small letters, split() cuts text into a list, startswith() checks the beginning, removeprefix() cuts a known start, join() glues a list into text.

```
branch = "  origin/release/16.5.9 "
branch.strip()                      # "origin/release/16.5.9"
branch.strip().split("/")[-1]       # "16.5.9"
"origin/test/x".removeprefix("origin/")   # "test/x"
```

f-strings

Put f before the quotes and you can place values inside { }.

```
return f"{release_version}.{next_build_number}"     # "16.5.9" + "." + "11"
```

Packages and imports

A folder of .py files is a package. import brings a function from another file into this one.

```
from pipeline.versioning import (
    get_next_build_version,
    get_beta_upload_name
)
```

Running other programs: subprocess.run

Starts a program, waits for it to finish and gives back its exit code (and its text, if captured).

```
result = subprocess.run(["git", "tag", "--list"], cwd=repo_path, capture_output=True, text=True)
if result.returncode != 0:          # 0 means it worked
    return []
return result.stdout.splitlines()   # one tag per line
```

Running other programs: subprocess.Popen

Starts a program but does NOT wait. The tool can keep drawing the screen while the program runs.

```
process = subprocess.Popen(command, cwd=cwd, stdout=subprocess.PIPE, stderr=subprocess.STDOUT)
...
if process.poll() is not None:      # None while it is still running
    ...
```

Threads and queues

A thread is a second worker inside the same program. One thread reads the output; the main thread draws. A queue is a safe box where one thread drops lines and the other picks them up.

```
output_queue = queue.Queue()
reader_thread = threading.Thread(target=read_output, args=(process, output_queue), daemon=True)
reader_thread.start()
```

Regular expressions (re)

A small pattern language for finding things in text, like "a number followed by %".

```
PERCENT_PATTERN = re.compile(r"(\d{1,3})\s*%")
match = PERCENT_PATTERN.search("Receiving objects:  45% (9/20)")
match.group(1)        # "45"
```

Redrawing one line: \r and colour codes

\r moves the cursor back to the start of the line, so the next print writes over it. Codes like \033[96m switch the colour; \033[0m switches it back.

```
print(f"\r  {CYAN}{spinner}{RESET}  {title} ...", end="", flush=True)
```

with open(...)

Opens a file and closes it again automatically, even if something goes wrong.

```
with open(output_log_path, "a", encoding="utf-8") as output_log:
    output_log.write(line + "\n")
```

16. Follow one build through the code

This table follows one Beta build from start to end and names every function it passes through. Open each file as you read the row.

#	What happens	Function	File
1	The tool starts	`main()`	`main.py`
2	Colours are turned on	`enable_console_colours()`	`ui/colours.py`
3	The banner is printed	`begin_msg()`	`ui/display.py`
4	Build type is asked	`get_build_type()` → `get_build_type_enum()`	`ui/input_handler.py`
5	Extensions are asked	`get_extension_list()` → `get_extension_enum()`	`ui/input_handler.py`
6	The branch is chosen	`get_required_release_branch()` or `get_test_branch()`	`workflow/release_flow.py`
7	Branches are listed	`get_release_branches()` / `get_Test_branches()`	`pipeline/git_operations.py`
8	The typed branch is checked	`validate_release_branch()` / `validate_test_branch()`	`pipeline/git_operations.py`
9	The version is read from the branch	`get_version_string_from_branch()` / `get_test_version_string_from_branch()`	`pipeline/git_operations.py`
10	Tags are read	`get_git_tags()`	`pipeline/git_operations.py`
11	The next version is worked out	`get_release_build_tags()` → `get_latest_build_number()` → `get_next_build_version()`	`pipeline/versioning.py`
12	Extensions become COP groups	`get_cop_extension_groups()`	`config/extension_config.py`
13	The upload name is made	`get_beta_upload_name()`	`pipeline/versioning.py`
14	Summary and confirmation	`show_build_summary()`, `get_build_confirmation()`	`ui/`
15	The 15 steps start	`run_beta_build()`	`workflow/beta_build_flow.py`
16	Each step heading	`show_step()`	`ui/display.py`

#	What happens	Function	File
17	Each Git or COP step	`checkout_release_branch()`, `cop_clean()`, … → `run_command_live()`	`pipeline/`
18	The moving line and the result	`show_progress()`, `show_result()`	`ui/progress.py`
19	The end	`show_build_success()` or `show_build_stopped()`	`ui/display.py`

17. How to make common changes (recipes)

Recipe A. Add a new extension

Say COP has a new group called [newext]. Four small edits are needed, and the number must match in each place.

1. `models/enums.py`: add `NEW_EXT=6` to `class Extension`.
2. `ui/display.py`: in `ask_extension_list_msg()` add `show_menu_option("6", "New Ext")`.
3. `ui/input_handler.py`: in `get_extension_enum()` add `elif this_input == "6" or this_input == "new ext":` and set `extension = Extension.NEW_EXT`.
4. `config/extension_config.py`: add `elif extension == Extension.NEW_EXT:` with `cop_groups.append("[newext]")`.

Recipe B. Add a new step to the Beta build

1. Write the command as one small function in `pipeline/`, using the runner:

```
def fetch_tags(repo_path: str) -> bool:
    """Downloads the newest tags from the server."""
    git_command = ["git", "fetch", "--tags", "--progress"]
    return run_command_live(git_command, repo_path, "GIT FETCH TAGS")
```

2. Import it in `workflow/beta_build_flow.py`.
3. Add a numbered comment block with `show_step()`, call the function, and stop with `show_build_stopped()` if it returns False.
4. Add 1 to `BETA_BUILD_STEPS` and renumber the steps after it.

Recipe C. Change a message or a colour

Only `ui/display.py` (messages and screens) or `ui/progress.py` (the moving line). Colours are in `ui/colours.py`. No other file needs to change.

Recipe D. Add a new build type (for example First Aid)

1. Make `workflow/first_aid_flow.py` with a function `run_first_aid_build(...)` that returns True or False, built like run_beta_build().
2. In `main.py`, replace the "not implemented yet" message in the `Build_Type.FIRST_AID` branch with a call to it.

Recipe E. Switch between test and real builds

Change `dev_mode` in `config/dev_config.py`. Nothing else.

18. Practice tasks

Try these in a copy of the project or on a test branch. Hints are given; answers are at the end of each task.

1 (easy) Change the banner text

Hint: `begin_msg()` in `ui/display.py`.

Answer: change the text passed to show_box(). Nothing else changes.

2 (easy) Add an extension to the menu only

Add "[6] Test" to the menu but nowhere else, run the tool and choose 6. What happens, and why?

Answer: "Incorrect extension entered." The menu is only text; get_extension_enum() does not know 6, so it returns Extension.NONE.

3 (medium) Download tags before the version is worked out

Hint: write `fetch_tags()` (recipe B) and call it in `main.py` just before `get_git_tags()`.

Answer: if fetch_tags() returns False, show an error and return. This also fixes issue 2 in section 11.

4 (medium) Show how many tags were found

Hint: `len(release_tags)` in main.py; pass it to `show_build_summary()` and add a row with `show_value()`.

Answer: add a new parameter to show_build_summary() and one show_value("Tags Found", str(count)) line.

5 (medium) Ask before uploading

Hint: copy `get_git_changes_push_confirmation()` and call the copy before step 15.

Answer: if the answer is no, show_build_stopped("Cancelled. Upload was not approved.") and return False.

6 (thinking) Why one second?

Why does `run_command_live()` stop one second after the program exits, instead of waiting for all its text?

Answer: cop make starts build workers that can keep the pipe open long after COP ends. Waiting for the pipe would freeze the tool.

7 (thinking) Why True / False?

Why do work functions return True or False instead of printing "failed" and carrying on?

Answer: so the workflow can stop at once and never push or upload after a failure.

19. Common mistakes and debugging tips

Mistake	What goes wrong	How to avoid it
Not checking a step's result	The build carries on after a failure and may push or upload something broken.	Always write `if not step(...):` then stop.
Using `print()` inside pipeline code	The moving line gets broken by stray text.	Use a ui function or let the runner print.
Leaving dev mode on	A "real" build runs on a test branch.	Check `config/dev_config.py` before real builds.
Menu, enum and COP map do not match	The wrong extension is built, or the choice is refused.	Change all four places in recipe A together.
Running in an IDE output panel	No animation, odd characters.	Use `run_build.bat` or a real console.

Mistake	What goes wrong	How to avoid it
Guessing why a step failed	Time lost.	Read the red box, then "Last output", then the log file. Run the grey > command by hand.

Tip. Functions in `pipeline/versioning.py` need no Git and no COP. You can try them in a Python shell: `get_next_build_version("16.5.9", 10)` gives `"16.5.9.11"`. This is the easiest place to start experimenting.

APPENDIX

Reference

Every function and every outside command, in one place.

A. Function reference

main.py

Function / name	What it does
`main()`	Runs the whole tool: questions, version, summary, build.

config/dev_config.py

Function / name	What it does
`dev_mode`	True = test branches, False = release branches.

config/extension_config.py

Function / name	What it does
`get_cop_extension_groups(extensions)`	HON → [hon], ALLSTEEL → [asi], GUNLOCKE → [gunlocke], HBF → [hbf], HNI_CORE → [hniCore].

models/enums.py

Function / name	What it does
`Build_Type`	NONE, BETA, FIRST_AID, PROJECT_BASED.
`Extension`	NONE, HON, ALLSTEEL, GUNLOCKE, HBF, HNI_CORE.

workflow/release_flow.py

Function / name	What it does
`get_required_release_branch(repo_path)`	Latest or custom release branch; asks again if it does not exist.
`get_test_branch(repo_path)`	Dev mode: lists test branches, asks for one, checks it.

workflow/beta_build_flow.py

Function / name	What it does
`run_beta_build(...)`	Runs the 15 steps; True if all worked.
`BETA_BUILD_STEPS`	15, used in the step headings.

pipeline/git_operations.py

Function / name	What it does
`get_release_branches(repo_path)`	Lists origin/release/* branches.
`get_version_string_from_branch(branch)`	Last part of a branch name.
`version_string_to_tuple(text)`	"16.5.9" → (16, 5, 9), so versions can be compared as numbers.
`get_latest_release_branch(branches)`	The branch with the highest version.
`validate_release_branch(branch, list)`	Accepts 3 forms; returns the full name or None.
`get_git_tags(repo_path)`	All tags.
`checkout_release_branch(branch, repo_path)`	git checkout (origin/ removed).
`pull_latest_changes(repo_path)`	git pull --progress.
`stage_changes(repo_path)`	git add .
`commit_build_version(version, repo_path)`	git commit -m <version>.
`create_build_tag(version, repo_path)`	git tag <version>.
`get_changed_files(repo_path)`	git status --short, one line per file.
`push_changes(repo_path)`	git push --progress.
`push_build_tag(version, repo_path)`	git push --progress origin <version>.
`get_Test_branches(repo_path)`	Lists origin/test/* branches.
`validate_test_branch(branch, list)`	Accepts 3 forms; keeps the real case; or None.
`get_test_version_string_from_branch(branch)`	Only the numbers: demo.build.17.5.0 → 17.5.0.

pipeline/cop_operations.py

Function / name	What it does
`cop_clean(repo_path)`	cop clean
`cop_repair(repo_path)`	cop repair
`cop_set_versions(groups, version, repo_path)`	cop setversions ... /notag /nocommit
`cop_make(groups, repo_path)`	cop make ...
`cop_upload(groups, name, repo_path)`	cop upload ... /uploadname=...

pipeline/versioning.py

Function / name	What it does
`get_release_build_tags(tags, version)`	Tags that start with "version.".
`get_latest_build_number(tags)`	Biggest last number, or 0.
`get_next_build_version(version, number)`	version + "." + (number + 1).
`get_beta_upload_name(build_version)`	Dots → dashes, + @BETA.

pipeline/command_runner.py

Function / name	What it does
`read_output(process, queue)`	Helper thread: cuts output into lines.
`run_command_live(command, cwd, title, detail)`	Runs one command with the live line; True if it worked.

ui/display.py

Function / name	What it does
`show_box / show_section / show_end_line`	Boxes, headings and lines.
`show_menu_option / show_value`	One menu row / one label-value row.
`prompt_text(text)`	The coloured ? prompt used with input().
`show_input_error_msg / show_error_msg / show_warning_msg`	! wrong answer, x error, ! warning.
`begin_msg / ask_build_type_msg / ask_extension_list_msg / ask_branch_info_msg`	Banner and menus.
`show_build_summary(...)`	The BUILD SUMMARY block.
`show_step(step, total, name)`	"STEP 3/15 · ..." heading.
`show_changed_files(files)`	Coloured git status list.
`show_build_stopped(reason) / show_build_success(...)`	The final red or green box.
`show_dev_mode_msg / ask_Test_branch_info_msg`	Dev mode line and test branch list.

ui/input_handler.py

Function / name	What it does
`get_build_type / get_build_type_enum`	Ask for and convert the build type.
`get_extension_list / get_extension_enum`	Ask for and convert the extensions.
`get_release_branch_input / get_custom_release_branch`	l or c, and the custom branch.
`get_build_confirmation`	Proceed with build?
`get_git_changes_confirmation / get_git_changes_push_confirmation`	Safe to commit? Safe to push?
`get_test_branch_input(list)`	Dev mode: which test branch?

ui/progress.py

Function / name	What it does
`can_animate()`	True if the output is a real console.
`format_elapsed_time(seconds)`	Seconds → HH:MM:SS.
`get_progress_bar(percent, frame)`	Filled bar, or the bouncing block.
`show_command / show_progress / clear_progress`	Grey > line, the moving line, clear it.
`show_error_header / show_error_line / show_target_result`	Lines that stay on screen.
`show_result / show_failure_details`	[OK] / [FAIL] and the details after a failure.

ui/colours.py

Function / name	What it does
`CYAN, GREEN, RED, YELLOW, DIM, BOLD, RESET, CLEAR_LINE`	Console colour codes.
`enable_console_colours()`	Turns colours on in a cmd window.

B. Every outside command the tool runs

Command	Where it is run from	Used for
`git branch -r --list origin/release/*`	`get_release_branches()`	list release branches
`git branch -r --list origin/test/*`	`get_Test_branches()`	list test branches (dev mode)
`git tag --list`	`get_git_tags()`	find used build numbers
`git checkout <branch>`	`checkout_release_branch()`	step 1
`git pull --progress`	`pull_latest_changes()`	step 2
`cop clean`	`cop_clean()`	step 3
`cop repair`	`cop_repair()`	step 4
`cop setversions [g] /version=X /notag /nocommit`	`cop_set_versions()`	step 5
`git status --short`	`get_changed_files()`	step 6
`git add .`	`stage_changes()`	step 8
`git commit -m <version>`	`commit_build_version()`	step 9
`git tag <version>`	`create_build_tag()`	step 10
`git push --progress`	`push_changes()`	step 12
`git push --progress origin <version>`	`push_build_tag()`	step 13
`cop make [g]`	`cop_make()`	step 14
`cop upload [g] /uploadname=<name>`	`cop_upload()`	step 15

All of them run inside the workspace `C:\CetDev\version16.5\extensions\custom`. The read-only Git commands (branch, tag, status) use subprocess.run directly; every step that changes something goes through run_command_live().